Leetcode 28 find the index of the first Occurence in a String
BruteForce

```python
class Solution:
    def strStr(self, haystack: str, needle: str) -> int:
        n= len(haystack)
        m= len(needle)
        for i in range(n-m +1):
            j=0
            while j<m and haystack[i+j]== needle[j]:
                j+=1

            if j==m:
                return i
        return -1
```

Brute force

haystack = "s a d but sad"

needle = "sad"

$n = 9$
$m = 3$ | $n - m = 6$

$i = 0$

haystack[0] = s , needle[0] = s
haystack[a] = a , needle[a] = a.

haystack[2] = d, needle[2] = d ✓

j == m

Return index

0

So the algorithm stops immediately

Visualization

Sad but sad

sad

Answer = 0

Time complexity O(n x m)

Space complexity -> O(1)

Optimal

```python
class Solution:
    def strStr(self, haystack: str, needle: str) -> int:
        if not needle:
            return 0

        # Build LPS array
        lps = [0] * len(needle)
        length = 0
        i = 1

        while i < len(needle):
            if needle[i] == needle[length]:
                length += 1
                lps[i] = length
                i += 1
            else:
                if length != 0:
                    length = lps[length - 1]
                else:
                    lps[i] = 0
                    i += 1

        # KMP search
        i = j = 0
        while i < len(haystack):
            if haystack[i] == needle[j]:
                i += 1
                j += 1

            if j == len(needle):
                return i - j

            elif i < len(haystack) and haystack[i] != needle[j]:
                if j != 0:
                    j = lps[j - 1]
                else:
                    i += 1

        return -1
```

KMP has two phases:

1. Builds LPS Array
2. Perform Pattern Matching

Step - 1 Build LPS Array

Sad

Initialize,

lps = [0, 0, 0]

length = 0

i = 1

needle[1] = 0

needle[0] = s

mismatch

lps[1] = 0

move i

i = 2

needle[2] = d

needle[0] = s

mismatch

lps[2] = 0

final LPS

[0, 0, 0]

Step - 2 Pattern matching

i -> haystack

j -> needle

Start

i = 0

j = 0

Step - 1

haystack[0] = s

needle[0] = s

match

i = 1 , j = 1

haystack[1] = a , needle[1] = a

Match

$i = 2$, $j = 2$

haystack[2] = d
needle[2] = d

Match

$i = 3$
$j = 3$

Now, $j == \text{len(needle)}$

$i - j = 3 - 3 = 0$

return 0

Time complexity → $O(m+n)$

Space complexity → $O(n)$